ESCUELA COLOMBIANA DE INGENIERÍA
DESARROLLO ORIENTADO POR OBJETOS
Excepciones
2026-2 — Laboratorio 4/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Perfeccionar el diseño y código de un proyecto considerando casos especiales y errores.
2. Construir clases de excepción encapsulando mensajes.
3. Manejar excepciones considerando los diferentes tipos.
4. Registrar los errores relevantes para el equipo de desarrollo de una aplicación en producción.
5. Utilizar los programas básicos de java (javac, java, javadoc, jar), desde la consola.
6. Experimentar las prácticas

   Designing. Simplicity.
   Designing. Refactor whenever and wherever possible.

ENTREGA

- Incluyan en un archivo `.zip` los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ)
- Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Reto: Desarrollar el catalogo de **NeoFlix**: una plataforma de *streaming* de nueva generación

La plataforma **neoFlix** ofrece tres tipos de contenidos: películas, series y colecciones. Una película es una producción audiovisual independiente, una serie es una producción compuesta por múltiples episodios y una colección es una agrupación de contenidos relacionados por una temática común. Para orientar a los usuarios sobre los contenidos que podrían resultar de su interés, **NeoFlix** maneja dos indicadores

- Popularidad: mide que tan atractiva es el contenido para los usuarios.
  ¿Qué porcentaje de quienes lo inician lo terminan?
- Valoración: mide la calidad percibida por los usuarios. [0..10]
  ¿Qué tan bueno consideran que es?

Las unidades audiovisuales (películas y episodios) almacenan los datos básicos para calcular los indicadores.

- Popularidad: (terminados/intentos) [0..100]
- Valoración: (sumaVotos/votos) [0..10]

Los indicadores de los otros contenidos normalmente se calculan como el promedio de los indicadores de los contenidos que los componen.



PARTE I. Practicando con excepciones

A MDD y BDD con `neoFlix` [En lab04.pdf, neoFlix.astah, neoFlix.*.java]

En este punto vamos a aprender a diseñar, codificar y probar usando excepciones. Para esto se van a trabajar algunos métodos de la clase `Serie`.

1. En su directorio descarguen los archivos contenidos en `neoFlix.zip`, revisen el contenido y estudien el diseño estructural de la aplicación (únicamente la zona en azul).
2. Expliquen por qué el proyecto no compila. Realicen las adiciones necesarias para lograrlo.
3. Dado el diseño y las pruebas, documenten (si es necesario) y codifiquen el método `raiting()`.
4. Dada la documentación y el diseño, codifiquen y prueben el método `raiting(default)`.
5. Documenten, diseñen, codifiquen y prueben el método `raiting(previous)`.
6. Propongan el cálculo, documenten, diseñen, codifiquen y prueben el método `popularity()`. (Definan reglas que obliguen a utilizar excepciones en sus cuatro momentos: propagar, lanzar y atender)

PARTE II. Desarrollando neoFlix

A Revisando línea base [En lab04.pdf]

1. En su directorio descarguen los archivos contenidos en neoFlixMore.zip, revisen el contenido. (a) ¿Cuántos archivos se tienen? (b) ¿Cómo están organizados? (c) ¿Cómo deberían estar organizados?
2. Estudien el diseño del programa: diagramas de paquetes y de clases. (a) ¿Cuántos paquetes tenemos? (b) ¿Cuántas clases tiene el sistema? (c) ¿Cómo están organizadas? (d) ¿Cuál es la clase ejecutiva?
3. Incluya los nuevos archivos en los directorios correspondientes para ejecutar el proyecto, (a) ¿Qué clases deben tener cada directorio? (c) ¿Cómo los incluyeron?
4. Ejecute el proyecto, (a) ¿Qué funcionalidades ofrece? (b) ¿Cuáles funcionan?
5. Revisen el código y la documentación del proyecto. (a) ¿De dónde salen los datos iniciales? (b) ¿Qué clase pide que se adicionen? (c) ¿Qué clase los adiciona?

B Realizando inversa. Adicionar y listar. Todo OK. [En lab04.pdf, neoFlix.astah, neoFlix.*.java]

(NO OLVIDEN BDD – MDD)

El objetivo es realizar ingeniería inversa a las funciones de adicionar y listar.

1. Adicionen un nuevo episodio y un nueva nueva.

   Serie
   The Neo Ninja
   Episodio The Hidden Village (a) ¿Qué ocurre? (b) ¿Cómo lo comprueban? Capturen la pantalla. (c) ¿Es adecuado este comportamiento?
2. Revisen el código asociado a adicionar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
3. Realicen ingeniería inversa para la capa de dominio para adicionar. Capturen los resultados de las pruebas de unidad.
4. Revisen el código asociado a listar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
5. Realicen ingeniería reversa para la capa de dominio para listar. Capturen los resultados de las pruebas de unidad.
6. Propongan y ejecuten una prueba de aceptación. Capturen las pantallas correspondientes.

C Fortaleciendo. Adicionar robusta. [En lab04.pdf, neoFlix.astah, neoFlix.*.java]

(NO OLVIDEN BDD – MDD)

El objetivo es fortalecer la funcionalidad de adicionar para hacerla más robusta. Para cada uno de los siguientes casos realice los pasos del 1 al 4.

(i) ¿Y si el nombre ya existe?
(ii) ¿Y si los valores numéricos no son numéricos?
(iii) ¿Y si los valores no son los esperados?
(suma de votos>10*votos o terminados>intentos)
(iv) Proponga una nueva condición

1. Propongan una prueba de aceptación que genere el fallo. (a) ¿Se generó alguna excepción? (b) ¿Por qué?
2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta: (a) ¿Qué método debería lanzar la excepción? (b) ¿Qué métodos deberían propagarla? (c) ¿Qué método debería atenderla? Expliquen claramente.
3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.
4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1. (a) ¿Qué sucede ahora? Capturen la pantalla.

D Registrando errores. Consultar por patrones. ¡Error! [En lab04.pdf, neoFlix.astah, neoFlix.*.java]

(NO OLVIDEN BDD – MDD)

El objetivo es aprender a manejar errores no esperados.

1. Consulten un contenido que inicia por T.(a) ¿Qué sucede? (b) ¿Qué error existe? Capturen el resultado. (d) ¿Quién debe conocer y quien NO debe conocer esta información?
2. Revisen el código asociado a consultar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
3. Realicen ingeniería reversa para la capa de dominio para consultar. No modifiquen el código. Capturen los resultados de las pruebas de unidad.
4. Exploren el método `record` de la clase `Log` (a) ¿Qué servicio presta?
5. Analicen el punto adecuado para que EN ESTE CASO se presente un mensaje especial de alerta al usuario, se guarde la información del error en el registro y continúe la ejecución. Expliquen y construyan la solución.
6. Ejecuten nuevamente la aplicación con el caso propuesto en 1. (a) ¿Qué mensaje salió en pantalla? (b) ¿La aplicación termina? (c) ¿Qué información tiene el archivo de errores?
7. Revisen el código, (a) ¿Qué otro caso puede generar un error? (b) ¿Cómo lo solucionarían?
8. (a) ¿Es adecuado que la aplicación continúe su ejecución después de sufrir un incidente como este? (b) ¿De qué dependería continuar o parar?
9. Modifiquen la aplicación para garantizar que SIEMPRE que haya un error no esperado en presentación se maneje de la forma adecuada. (a) ¿Cuál fue la solución seleccionada?

PARTE III. DE BLUEJ A CONSOLA

En esta sección vamos a aprender a usar java desde consola. Para esto vamos a trabajar con el proyecto `neoFlix`.

A Comandos básicos del sistema operativo [En lab04.pdf]

Antes de iniciar debemos repasar los comandos básicos del manejo de la consola.

1. Investiguen los comandos para moverse en la estructura de directorios: crear, borrar, listar su contenido y copiar o eliminar un archivo.
2. Organicen un nuevo directorio con la estructura propuesta para probar desde allí los comandos de consola. Consulten y capturen el contenido de su directorio.

```
neoFlix
  src
    domain
    presentation
    test
```

3. En el directorio `src/domain` copien únicamente el archivo `Serie.java` del paquete de dominio. Consulten y capturen el contenido de `src/domain`

B Estructura de proyectos java [En lab04.pdf]

En java los proyectos se estructuran considerando tres directorios básicos.

```
neoFlix
  src
  bin
  doc
  lib
```

1. Investiguen los archivos que deben quedar en cada una de esas carpetas y la organización interna de cada una de ellas.
2. (a) ¿Qué archivos de `neoFlix` deberían estar en el directorio `src`? (b) ¿Cómo deberían estar organizados?
3. Copien a `src` únicamente los archivos de los paquetes de presentación y dominio. Consulten y capturen el contenido de los directorios que se modificaron.

C Comandos de java [En lab04.pdf]

1. Consulten para qué sirven cada uno de los siguientes comandos:
   `javac`
   `java`
   `javadoc`
   `jar`
2. Creen una sesión de consola y consulten en línea las opciones de los comandos `java` y `javac`. Capturen las pantallas.
3. Busquen la opción que sirve para conocer la versión a que corresponden estos dos comandos. Documenten el resultado.

D Compilando [En lab04.pdf]

1. Utilizando el comando `javac`, desde el directorio raiz (desde `neoFlix` con una sola instrucción), compile el proyecto. (a) ¿Qué instrucción completa tuvo que dar a la consola para compilar TODO el proyecto? Tenga presente que se pide un único comando y que los archivos compilados deben quedar en los directorios respectivos.
2. Revise de nuevo el contenido del directorio de trabajo y sus subdirectorios. (a) ¿Cuáles nuevos archivos aparecen ahora y dónde se ubican? Capturen las pantallas con los contenidos de los

directorios de `neoFlix/bin`.

E Documentando [En lab04.pdf]

1. Utilizando el comando `javadoc`, desde el directorio raiz, genere la documentación (API) en formato `html`, en este directorio. (a) ¿Cuál es el comando completo para generar esta documentación?
2. (a) ¿Cuál archivo hay qué abrir para empezar a navegar por la documentación? Ábranlo y capturen la pantalla.

F Ejecutando [En lab04.pdf]

1. Empleando el comando java, desde el directorio raíz, ejecute el programa. (a) ¿Cómo utilizó este comando?
2. Capturen la pantalla del programa en ejecución.

G Probando [En lab04.pdf]

1. Adicione ahora los archivos del paquete de pruebas y trate de compilar nuevamente el programa. (a) ¿Qué pasa?
2. Tenga en cuenta que estas clases requieren la librería junit. (a) ¿Dónde se incluye una librería en la estructura de archivos? (b) ¿Cómo se compila incluyendo un `.jar`? (c) ¿Qué instrucción completa tuvo que dar a la consola para compilar?
3. Ejecute desde consola las pruebas. (a) ¿Cómo utilizó el comando `java`?.
4. Pegue en su documento el resultado de las pruebas

H Empaquetando [En lab04.pdf]

1. Consulten como utilizar desde consola el comando `jar` para empaquetar su programa entregable en un archivo `.jar`, que contenga los archivos `bytecode` necesarios (no las fuentes ni las clases de prueba)(a) ¿Cómo empaquetó jar ?
2. (a) ¿Cómo se ejecuta el proyecto empaquetado? Capturen la pantalla del programa en ejecución.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.